Skill-Based Resume Ranking System with Explainable AI for Recruitment Automation

Tanjina Akter Nisa
Dept. of Computer Science and Engineering
University of Asia Pacific
Dhaka, Bangladesh
22201086@uap-bd.edu

Md. Daudul Islam
Dept. of Computer Science and Engineering
University of Asia Pacific
Dhaka, Bangladesh
22201088@uap-bd.edu

Saswata Ghosal
Dept. of Computer Science and Engineering
University of Asia Pacific
Dhaka, Bangladesh
22201090@uap-bd.edu

***Abstract*—Manual resume screening in recruitment is time-consuming, inconsistent, and susceptible to human bias. This paper presents a Skill-Based Resume Ranking System that automates candidate evaluation using a combination of Machine Learning (ML) and Deep Learning (DL) models. The system accepts PDF resumes and job requirements as input, extracts structured information using Natural Language Processing (NLP) techniques, and computes a hybrid match score for each candidate. Four models are employed: Logistic Regression and Random Forest for job category classification using TF-IDF features, DistilBERT fine-tuned on resume text for deep contextual classification, and Sentence-BERT (SBERT) for semantic similarity scoring between CV content and job descriptions. The hybrid scoring formula integrates semantic similarity (40%), skill match (30%), experience (15%), and education (15%) to produce a transparent, explainable ranking. All three classification models achieved F1 scores of 0.99, with Logistic Regression and Random Forest reaching 99.48% accuracy and DistilBERT reaching 99%. The system features a Streamlit web interface with HR login, batch PDF upload, ranked candidate output, percentile analytics, and CSV report export.**

***Index Terms*—Resume Screening, Machine Learning, Deep Learning, Natural Language Processing, DistilBERT, SBERT, TF-IDF, Explainable AI, Recruitment Automation**

I. Introduction

Recruitment is one of the most resource-intensive functions in human resource management. Organizations receive hundreds of applications for a single job posting, and manually reviewing each resume requires significant time and effort from HR professionals. Studies indicate that HR teams spend up to 72% of their working time on resume screening, with each CV receiving an average of only 6–10 seconds of human attention [1]. This rushed process often leads to inconsistent evaluations, unintentional bias, and the risk of overlooking qualified candidates.

The advent of NLP and transformer-based language models presents an opportunity to address these challenges through intelligent automation. Recent developments in pre-trained models such as BERT [2], DistilBERT [3], and Sentence-BERT (SBERT) [4] have enabled machines to understand the semantic content of text with high accuracy, making them well-suited for resume analysis tasks.

This paper proposes an end-to-end AI-powered resume screening system that combines ML and DL models in a hybrid pipeline. The system extracts information from PDF CVs, classifies resumes by job category, computes a multi-dimensional match score, and ranks candidates with full explainability. The key contributions of this work are:

- A hybrid scoring formula combining SBERT semantic similarity with rule-based skill, experience, and education matching.
- A comparative evaluation of two ML models (Logistic Regression, Random Forest) and two DL models (DistilBERT, SBERT) on the same resume classification task.
- An Explainable AI (XAI) output system providing matched skills, missing skills, score breakdowns, and improvement suggestions.
- A functional Streamlit web application enabling HR professionals to upload CVs, set job requirements, and receive ranked candidate results with downloadable reports.

II. Dataset and Preprocessing

A. Dataset

The UpdatedResumeDataSet was obtained from Kaggle [7]. It contains 962 real resume samples labeled across 25 distinct job categories. Each record consists of two fields: the resume text (unstructured free-form content) and the job category label. The dataset covers a wide range of professional domains, ensuring diversity in training samples. Table I presents the distribution of selected categories.

B. Preprocessing Steps

The following preprocessing pipeline was applied to prepare data for model training:

1) **Null Value Removal:** Rows with missing values were removed using `df.dropna()`.

TABLE I: Dataset Category Distribution (Selected)

Job Category	Sample Count
Java Developer	84
Testing	70
DevOps Engineer	55
Python Developer	48
Web Designing	45
HR	44
Data Science	40
Blockchain	40
Mechanical Engineer	40
Other (16 categories)	≈556
Total	**962**

2) **Label Encoding:** The 25 category strings were converted to integer labels (0–24) using scikit-learn's `LabelEncoder`.
3) **Train/Validation Split:** The dataset was divided 80/20 (769 training, 193 validation) using stratified sampling.
4) **TF-IDF Vectorization (ML models):** `TfidfVectorizer` with `max_features=5000` and `ngram_range=(1,2)` converted resume text into sparse numeric feature matrices [6].
5) **DistilBERT Tokenization (DL model):** `DistilBertTokenizerFast` tokenized text with `max_length=256`, padding, and truncation.
6) **PDF Text Extraction:** The `pdfplumber` library extracted raw text from uploaded PDF files, followed by regex normalization.
7) **Skill and Information Extraction:** A regex-based pipeline extracted years of experience, education level, and skills using keyword matching with fuzzy normalization.

III. System Design and Methodology

A. System Architecture Overview

The proposed system follows a multi-stage pipeline: PDF ingestion and text extraction, structured information extraction, model-based analysis, hybrid score computation, and ranked output generation. The architecture integrates four models operating on different tasks.

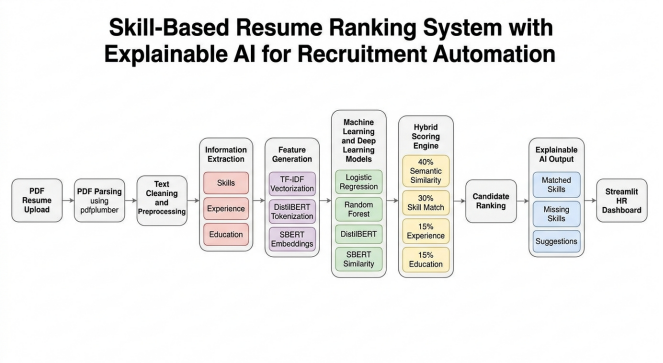

Fig. 1: Overall system architecture of the proposed resume ranking system.

Figure 1 illustrates the overall system workflow.

B. ML Model 1 – Logistic Regression

Logistic Regression with TF-IDF features serves as the primary ML baseline. Given the high-dimensional, sparse nature of TF-IDF vectors, logistic regression is well-suited to linear separability in text classification tasks [8]. The model was configured with `max_iter=1000` and regularization parameter `C=1.0`.

C. ML Model 2 – Random Forest

Random Forest [5] is an ensemble method that aggregates predictions from 100 decision trees (`n_estimators=100`) trained on random subsets of TF-IDF features. Its inclusion enables comparison of a non-linear ensemble approach against the linear Logistic Regression model.

D. DL Model 1 – DistilBERT

DistilBERT (`distilbert-base-uncased`) [3] is a distilled version of BERT with 67 million parameters, approximately 40% smaller and 60% faster than the original BERT while retaining 97% of its language understanding capability. The model was fine-tuned on the resume dataset for sequence classification with 25 output labels. Fine-tuning configuration is as follows:

- Epochs: 3; Batch size: 8; Optimizer: AdamW
- Weight decay: 0.01; Warmup steps: 5
- Max token length: 256
- Evaluation strategy: per epoch with best model loading

DistilBERT captures contextual and semantic information that bag-of-words TF-IDF cannot. For instance, the phrase "developed scalable backend APIs" is understood in context rather than as isolated keywords.

E. DL Model 2 – SBERT (all-MiniLM-L6-v2)

Sentence-BERT [4] generates dense 384-dimensional sentence embeddings optimized for semantic similarity tasks. The pre-trained `all-MiniLM-L6-v2` model encodes both the CV text and the job description into embedding vectors. Cosine similarity between these vectors measures semantic alignment:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \quad (1)$$

where A and B are the CV and job description embedding vectors respectively.

F. Hybrid Scoring Formula

The final candidate score is computed using a weighted combination of four components:

$$S = 0.40 \cdot s_{sem} + 0.30 \cdot s_{skill} + 0.15 \cdot s_{exp} + 0.15 \cdot s_{edu} \quad (2)$$

where s_{sem} is the SBERT cosine similarity score, s_{skill} is the proportion of required skills matched, $s_{exp} = \min(\text{CV\_years}/\text{required\_years},\ 1.0)$, and s_{edu} is the education match score (1.0 if met, 0.5 if one level below, 0.0 otherwise). SBERT receives the highest weight at 40% as it captures the overall semantic relevance of the CV, even when exact keywords are absent.

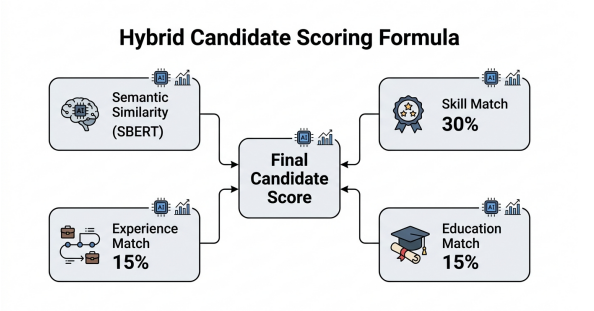

Fig. 2: Weighted hybrid scoring mechanism used for candidate ranking.

Figure 2 visualizes the weighted hybrid scoring mechanism. Table II summarizes score interpretation.

TABLE II: Score Interpretation

Score Range	Verdict	Action
75%–100%	Highly Recommended	Shortlist
55%–74%	Moderate Match	Further review
0%–54%	Not Recommended	Skill gaps exist

G. NLP Pipeline

The complete NLP pipeline comprises seven stages: (1) PDF text extraction, (2) text normalization, (3) skill extraction with fuzzy matching, (4) experience extraction via regex, (5) education level detection, (6) TF-IDF vectorization or DistilBERT tokenization, and (7) SBERT encoding with cosine similarity computation.

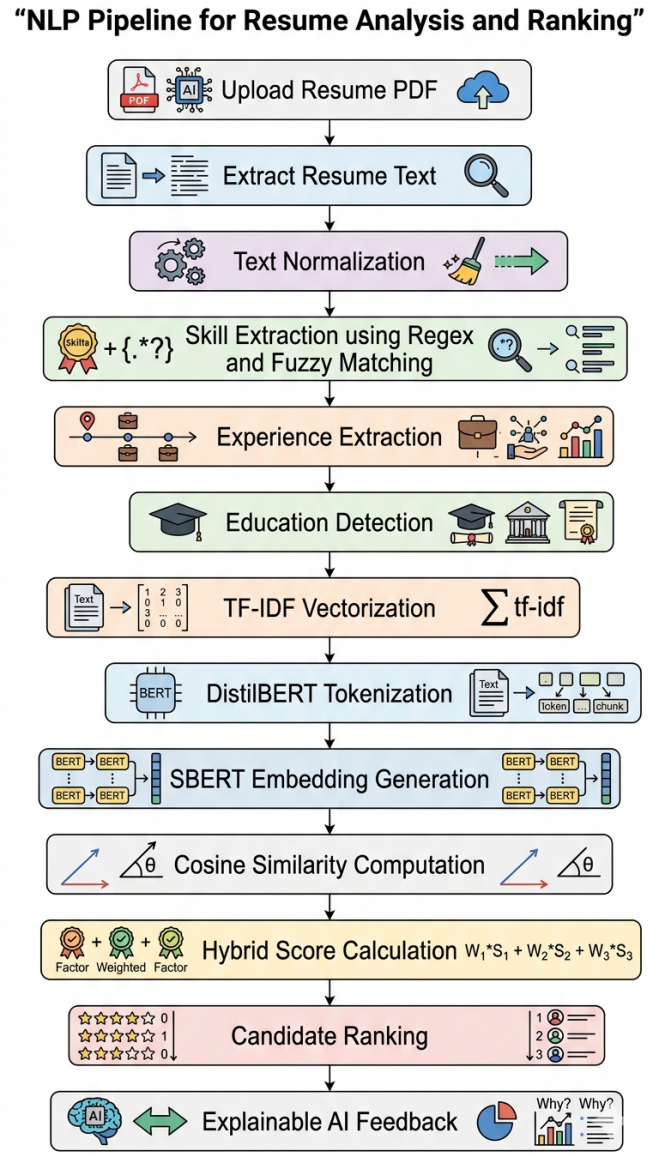

Fig. 3: NLP processing pipeline for resume analysis and ranking.

Figure 3 shows the detailed NLP processing stages.

IV. Implementation and Results

A. Implementation Setup

The system was implemented in Python 3.12 on a Windows CPU environment. Table III lists all tools and frameworks used.

B. Classification Results

All three classification models were evaluated on the 193-sample validation set. Results are shown in Table IV.

TABLE III: Tools and Frameworks

Component	Tool	Version
Core Language	Python	3.12
ML Models	scikit-learn	1.8.0
DL Framework	PyTorch	2.3
Transformers	HuggingFace	5.6.2
SBERT	sentence-transformers	5.4.1
PDF Extraction	pdfplumber	0.11.9
Web Interface	Streamlit	1.57.0
Data Handling	pandas, numpy	3.0.2
Visualization	matplotlib	Latest

TABLE IV: Model Performance Comparison

Model	Type	Acc.	P	R	F1
Logistic Regression	ML	99.48%	1.00	0.99	0.99
Random Forest	ML	99.48%	0.99	0.99	0.99
DistilBERT	DL	99.00%	0.99	0.99	0.99
SBERT (MiniLM)	DL	N/A	—	—	—

P: Precision, R: Recall, F1: F1 Score
SBERT performs similarity scoring, not classification.

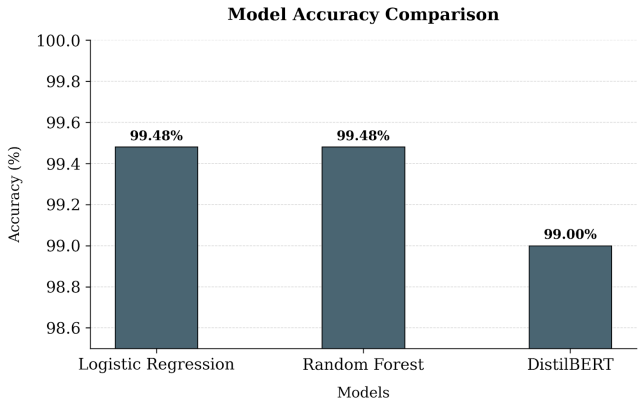

Fig. 4: Accuracy comparison among ML and DL models.

Figure 4 presents a visual comparison of model accuracies.

C. DistilBERT Training Progress

Training and validation loss decreased consistently over three epochs, confirming effective learning without overfitting. Table V presents the training history.

TABLE V: DistilBERT Training and Validation Loss

Epoch	Training Loss	Validation Loss
1	2.005	1.715
2	0.673	0.579
3	0.385	0.342

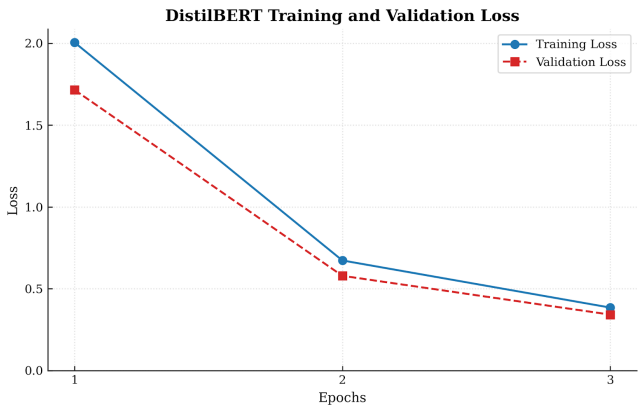

Fig. 5: Training and validation loss across DistilBERT fine-tuning epochs.

As shown in Figure 5, both training and validation loss decreased steadily, confirming that the model learned effectively without overfitting.

D. Output System

The system provides two output modes. **Single CV Mode** delivers a match score (%), matched skills, missing skills, four-component score breakdown, AI-generated score explanation, and specific improvement suggestions. **Multiple CV Mode** includes all single CV outputs plus candidate ranking with medals, percentile grouping (Top 10%, Middle 50%, Bottom 50%), group insights showing common strengths and weaknesses, a comparative score breakdown chart, and CSV report download.

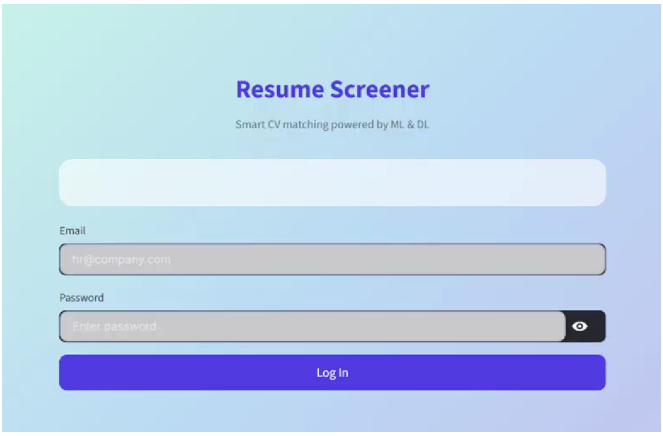

Fig. 6: Streamlit-based HR dashboard showing ranked candidate output.

Figure 6 demonstrates the user-friendly Streamlit interface.

V. Discussion

A. Why High Accuracy?

The 99%+ accuracy across all classification models is attributed to the inherently distinct vocabulary of the 25 job categories. A Java Developer resume contains predominantly

Java-specific terminology (Spring Boot, JPA, Hibernate) that rarely appears in an HR resume, and vice versa. TF-IDF effectively captures these category-distinguishing terms, making linear classification highly effective. DistilBERT achieves similar performance by learning contextual representations that align with category boundaries.

Importantly, the consistent performance across both training and validation sets confirms that the high accuracy reflects genuine category separability rather than overfitting. Both training and validation losses decreased steadily across all three epochs, with no divergence between the two curves.

B. ML vs. DL Comparison

Both ML models (Logistic Regression and Random Forest) achieved 99.48% accuracy — marginally higher than DistilBERT's 99%. This demonstrates that for clearly separable text categories with distinct vocabularies, simpler ML approaches can match or exceed DL performance at a fraction of the computational cost. Logistic Regression completed training in approximately 2 seconds, while DistilBERT required approximately 30 minutes on CPU.

However, DistilBERT's advantage lies in its ability to understand context and semantic nuance — critical for the SBERT-based semantic matching component that addresses the core ranking task beyond simple classification. When candidates have overlapping skills or non-standard CV language, SBERT's contextual understanding provides more accurate matching than keyword-based approaches.

C. Limitations

- The dataset is limited to 962 English-language resumes across 25 categories.
- Experience extraction relies on regex patterns and may miss non-standard phrasings.
- DistilBERT training on CPU is slow ($\sim$30 min); GPU deployment would improve performance.
- SBERT scores may be lower for very short or poorly formatted CVs.
- Authentication uses plaintext comparison, not suitable for production deployment.

D. Future Improvements

- Implement Named Entity Recognition (NER) for more robust skill and experience extraction.
- Expand dataset with more diverse, real-world resume samples.
- Add GPU-based training and cloud deployment.
- Integrate larger language models (LLaMA, GPT-4) for richer semantic understanding.
- Add multi-language support for non-English CVs.

VI. Conclusion

This paper presented an AI-powered resume screening and ranking system combining Machine Learning and Deep Learning in a hybrid pipeline. The system employs Logistic Regression and Random Forest with TF-IDF features for efficient job category classification, DistilBERT for deep contextual classification, and SBERT for semantic CV-to-job matching. A weighted hybrid scoring formula integrates semantic similarity, skill match, experience, and education into a single transparent score.

All three classification models achieved F1 scores of 0.99, with ML models reaching 99.48% and DistilBERT reaching 99% accuracy. The comparative evaluation demonstrates that while DL models offer richer semantic understanding, ML models remain competitive for structured text classification with distinct categories.

The Streamlit web application provides a functional prototype suitable for HR use, featuring batch PDF upload, ranked candidate output, explainable AI insights, percentile analytics, and CSV export. Future work will focus on dataset expansion, NER-based extraction, GPU deployment, and integration of larger language models for enhanced semantic reasoning.

Acknowledgment

The authors would like to thank Assistant Professor Faria Zarin Subah of the Department of Computer Science and Engineering, University of Asia Pacific, for her guidance and support throughout this project.

References

[1] S. Nawaz and A. Gomes, "AI in Human Resource Management: A Review of Resume Screening Automation," *Int. J. Emerg. Technol. Innov. Res.*, vol. 6, no. 5, 2019.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, Hong Kong, China, 2019, pp. 3982–3992.

[5] L. Breiman, "Random Forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[6] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988.

[7] Gaurav Duttakiit, "UpdatedResumeDataSet," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset. [Accessed: May 2026].

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.